Plant Disease Detection

Section 1

Data Preprocessing

TensorFlow | OpenCV | Keras | Sklearn | Pandas

1. Load Images
2. Labels
3. Balance
4. Split
5. Generators
6. Preprocess

1

📦 Load Images from PlantVillage

Collect all .jpg files and separate them into healthy vs diseased lists

```
import os
import glob as gb
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
from tqdm import tqdm
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import tensorflow as tf
import keras
import kagglehub
path = kagglehub.dataset_download(
    "abdallahalidev/plantvillage-dataset")

color_path = os.path.join(
    path, "plantvillage dataset/color")
)
# Collect images by category
healthy_images  = []
diseased_images = []

for folder in os.listdir(color_path):
    folder_path = os.path.join(color_path, folder)
    if os.path.isdir(folder_path):
        files = gb.glob(os.path.join(
            folder_path, '*.[jJ][pP][gG]'))
        if 'healthy' in folder.lower():
            healthy_images.extend(files)
        else:
            diseased_images.extend(files)
```

📁 What it does

- → Downloads dataset via kagglehub
- → Scans the /color directory
- → Uses glob to collect .jpg files
- → Splits into 2 lists by folder name

🔑 Key Functions

`os.listdir()`	list all folders
`glob()`	collect image files
`extend()`	append to list

📊 Output

```
healthy_images  →  List[str]
diseased_images →  List[str]
```

2 🏷️ Create DataFrame & Labels

Extract labels from folder names and build a structured DataFrame

```python
def get_label_from_path(img_path):
    folder_name = os.path.basename(
        os.path.dirname(img_path)
    )
    return 'healthy' if 'healthy' in folder_name.lower()
           else 'diseased'

all_paths  = healthy_images + diseased_images
all_labels = [get_label_from_path(p) for p in all_paths]

df = pd.DataFrame({
    'image_path': all_paths,
    'label':      all_labels
})
```

💡 Label Logic

Folder name contains 'healthy'

→ label = 'healthy'

Otherwise:

→ label = 'diseased'

Case-insensitive check (.lower())

📋 DataFrame Columns

`image_path`	Full path to image file
`label`	'healthy' or 'diseased'

DataFrame Preview (df.head())

#	image_path	label
0	.../Tomato_healthy/img1.jpg	healthy
1	.../Corn_Northern_Leaf_Blight/img2.jpg	diseased
2	.../Pepper_healthy/img3.jpg	healthy

📊 Output

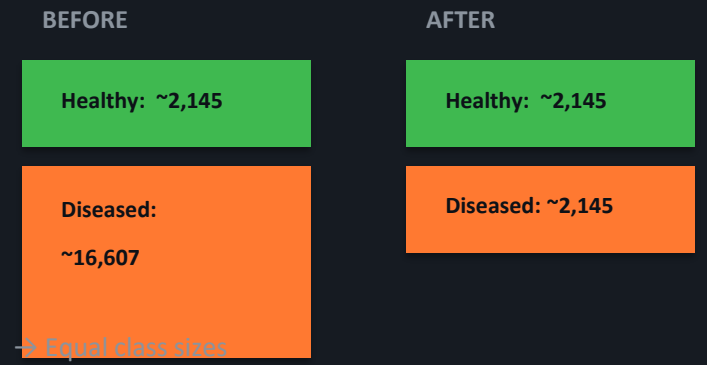
`df → pd.DataFrame (image_path, label)`

3 ⚖️ Balance the Dataset

Undersample diseased class to match healthy count — prevents model bias

```
df_healthy  = df[df['label'] == 'healthy']
df_diseased = df[df['label'] == 'diseased']

min_size = len(df_healthy)

df_diseased_balanced = df_diseased.sample(
    n=min_size,
    random_state=42
)

df_balanced = pd.concat([
    df_healthy,
    df_diseased_balanced
])

df_balanced = df_balanced.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)
```

📊 Before vs After Balancing



→ Equal class sizes
→ random_state=42 for reproducibility
→ frac=1 shuffles the full dataset

⚠️ Why balance?

Imbalanced data causes the model to ignore minority class (healthy)

📊 Output

`df_balanced` → `pd.DataFrame` (equal healthy & diseased rows)

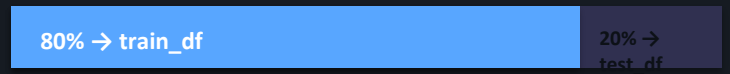
4 ✂️ Train / Validation / Test Split

80% Train — 20% Test, then 85% Train — 15% Validation from the training set

```python
from sklearn.model_selection import train_test_split

# 80% train  –  20% test
train_df, test_df = train_test_split(
    df_balanced,
    test_size=0.2,
    random_state=42,
    stratify=df_balanced['label']
)

# From train: 85% train  –  15% validation
train_df, val_df = train_test_split(
    train_df,
    test_size=0.15,
    random_state=42,
    stratify=train_df['label']
)
```

📊 Split Diagram

df_balanced (100%)



train_df (80%)

Final Splits

train_df	val_df	test_df
~68%	~12%	~20%

✓ stratify ensures each split has equal healthy / diseased ratio

✓ random_state=42 → reproducible results every run

5 🔄 Create Image Generators

Use Keras ImageDataGenerator to rescale pixels and create batch pipelines

```
from tensorflow.keras.preprocessing.image import \
    ImageDataGenerator

train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen   = ImageDataGenerator(rescale=1./255)
test_datagen  = ImageDataGenerator(rescale=1./255)

BATCH_SIZE = 32
IMG_SIZE   = (128, 128)

train_generator = train_datagen.flow_from_dataframe(
    dataframe=train_df,
    x_col='image_path', y_col='label',
    target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='binary', shuffle=True
)
val_generator = val_datagen.flow_from_dataframe(
    dataframe=val_df,
    x_col='image_path', y_col='label',
    target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='binary', shuffle=False
)
test_generator = test_datagen.flow_from_dataframe(
    dataframe=test_df,
    x_col='image_path', y_col='label',
    target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='binary', shuffle=False
)
```

⚙️ Parameters Explained

`rescale=1./255`	Normalize pixels to [0,1]
`BATCH_SIZE=32`	Images processed per step
`IMG_SIZE=(128,128)`	Resize all images
`class_mode='binary'`	2-class output (0/1)
`shuffle=True`	Only for training data

🔢 Rescaling Formula

```
pixel_new = pixel_old / 255.0
```

Range: [0, 255] → [0.0, 1.0]

Required by neural networks

📊 Output

```
3 generators: train / val / test
Ready for model.fit()
```

6 🖼️ Preprocess a Single Image for Prediction

Manually apply the same preprocessing steps when predicting on a new image

```python
def preprocess_single_image(image_path):

    # Read image with OpenCV (returns BGR)
    img_bgr = cv2.imread(image_path)

    # Convert BGR → RGB
    img_rgb = cv2.cvtColor(
        img_bgr, cv2.COLOR_BGR2RGB
    )

    # Resize to 128×128 pixels
    img_resized = cv2.resize(img_rgb, (128, 128))

    # Normalize pixel values to [0, 1]
    img_normalized = img_resized / 255.0

    # Add batch dimension: (128,128,3) → (1,128,128,3)
    img_final = np.expand_dims(
        img_normalized, axis=0
    )

    return img_final
```

🔄 Processing Pipeline

cv2.imread()
Read .jpg → BGR array
→ BGR image

cvtColor()
BGR → RGB conversion
→ RGB image

cv2.resize()
Resize to 128×128
→ (128, 128, 3)

/ 255.0
Normalize to [0.0, 1.0]
→ float32 array

expand_dims()
Add batch axis
→ (1, 128, 128, 3)

CNN Model

Section 2

Convolutional Neural Network

1 Imports | 2 Architecture | 3 Block 1 | 4 Block 2 | 5 Block 3 | 6 Head | 7 Compile

1 📦 Imports

Load all required Keras layers and model class from TensorFlow

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
    Dropout, Dense, GlobalAveragePooling2D,
    BatchNormalization
```

📦 What We Import

Sequential → linear layer stack
Conv2D → 2D convolution
MaxPooling2D → spatial downsampling
BatchNorm → normalize outputs
Dropout → regularization
Dense → fully-connected layer
GlobalAvgPool → replaces Flatten

🔑 Why These Layers?

Conv2D extracts local features (edges, textures).
MaxPooling halves spatial size each block.
BatchNorm speeds training & stabilizes.
Dropout prevents overfitting.
GlobalAvgPool reduces params vs Flatten.

2 Build Model — Block 1 (32 filters)

Sequential model init → first Conv block: Conv → BN → Conv → Pool → Dropout

```python
model = Sequential()

# ── Block 1 ── 32 filters ──────────────────
model.add(Conv2D(32, (3,3), padding='same',
                 activation='relu',
                 input_shape=(128, 128, 3)))
model.add(BatchNormalization())
model.add(Conv2D(32, (3,3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Dropout(0.25))
```

Conv2D — 32 filters

kernel (3,3) → 3×3 sliding window
padding='same' → keep spatial size
activation='relu' → non-linearity
input_shape=(128,128,3) → RGB image

BatchNorm + Pool + Drop

BatchNorm → normalise after Conv
2nd Conv32 → deeper features
MaxPool(2,2) → 128→64 spatial
Dropout(0.25) → 25% neurons zeroed

2 Build Model — Block 2 (64 filters)

Second convolutional block — doubles filters to 64 for richer feature detection

```python
# ── Block 2 ── 64 filters ──────────────────
model.add(Conv2D(64, (3,3), padding='same',
                 activation='relu'))
model.add(BatchNormalization())
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Dropout(0.25))
```

Why double to 64?

Deeper layers need more filters to capture complex, abstract patterns.

32 filters → edges & textures
64 filters → shapes & object parts

Doubling is a standard CNN practice.

Spatial Size After Block 2

Input: 128×128×3
After B1 Pool: 64×64×32
After B2 Pool: 32×32×64

Each MaxPool(2,2) halves width & height.
Feature maps grow deeper, smaller in area.

2 Build Model — Block 3 (128 filters)

Third block — 128 filters, higher Dropout(0.3) for stronger regularisation

```python
# ── Block 3 ── 128 filters ──────────────────
model.add(Conv2D(128, (3,3), padding='same',
                 activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(2, 2))
model.add(Dropout(0.3))
```

Why only ONE Conv here?

Block 3 has one Conv2D instead of two. At 128 filters the model is already rich enough — adding a second Conv would increase cost without much benefit.

Dropout raised to 0.3 (was 0.25) because deeper layers overfit more easily.

Size After Block 3

After B2 Pool: 32×32×64
After B3 Pool: 16×16×128

Total params so far: large
→ GlobalAvgPool next will collapse 16×16×128 → 128 values

3 🎯 Classifier Head

GlobalAveragePooling replaces Flatten → Dense(128) → Dropout → sigmoid output

```python
# ── Classifier Head ──────────────────────────
model.add(GlobalAveragePooling2D())

model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

model.add(Dense(1, activation='sigmoid'))
```

🌐 GlobalAveragePooling2D

16×16×128 → 128-D vector
Takes mean of each feature map.
Replaces Flatten → far fewer params.
→ Less overfitting risk.

🔷 Dense(128) + Dropout(0.5)

Dense(128, relu) → learn combinations
Dropout(0.5) → 50% dropout, strong regularisation before output layer.

🟢 Output: Dense(1, sigmoid)

sigmoid → output in [0, 1]
0.0 → class 0 (e.g. Healthy)
1.0 → class 1 (e.g. Diseased)
Threshold at 0.5 for prediction.

4 ⚙ Compile & Summary

Configure optimizer, loss function, metrics — then print full architecture

```python
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

⚡ Optimizer: Adam

Adam = Adaptive Moment Estimation
Adjusts learning rate per parameter.
Default lr = 0.001
→ Converges fast & reliably.

📉 Loss: binary_crossentropy

Standard loss for binary (0/1) output.
Measures how far sigmoid output is from true label (0 or 1).
Minimised during backpropagation.

📊 model.summary()

Prints every layer name, output shape and parameter count.
Use it to verify architecture before starting training.

5 Full Code — Complete CNN Model

Every single line — nothing deleted — ready to present line by line

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout, Dense,
    GlobalAveragePooling2D, BatchNormalization
model = Sequential()
model.add(Conv2D(32, (3,3), padding='same', activation='relu',
                 input_shape=(128,128,3)))
model.add(BatchNormalization())
model.add(Conv2D(32, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Dropout(0.25))
model.add(Conv2D(64, (3,3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Dropout(0.25))
model.add(Conv2D(128, (3,3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(2,2))
model.add(Dropout(0.3))
model.add(GlobalAveragePooling2D())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])
model.summary()
```

Block Summary

Block 1: Conv×2 + BN + Pool + Drop(0.25)
Block 2: Conv×2 + BN + Pool + Drop(0.25)
Block 3: Conv×1 + BN + Pool + Drop(0.30)
Head: GAP → Dense(128) → Drop(0.5)
Output: Dense(1, sigmoid)

Filter Progression

32 → 32 → Block 1
64 → 64 → Block 2
128 → Block 3
128-D after GlobalAvgPool
1 output unit (sigmoid)

🚀 Model Training

Section 3

PlantVillage — Healthy vs Diseased

1 Generators | 2 Architecture | 3 Callbacks | 4 Training | 5 Visualization | 6 Evaluation | 7 Comparison

)

ANN Model • Python / TensorFlow • Binary Classification

1 🚀 Model Training

model.fit() — trains CNN on batches for 5 epochs with callbacks

```
# CNN Training
checkpoint = ModelCheckpoint(
    "/content/drive/MyDrive/best_model.h5",
    monitor='val_accuracy', save_best_only=True, verbose=1
)
early_stop = EarlyStopping(
    monitor='val_accuracy', patience=5, restore_best_weights=True
)
reduce_lr = ReduceLROnPlateau(
    monitor='val_accuracy', factor=0.5, patience=3, verbose=1, min_lr=1e-6
)
train_gen = train_datagen.flow_from_dataframe(
    train_df, x_col="image_path", y_col="label",
    target_size=(128,128), class_mode="binary", batch_size=32, shuffle=True
)
val_gen = val_datagen.flow_from_dataframe(
    val_df, x_col="image_path", y_col="label",
    target_size=(128,128), class_mode="binary", batch_size=32, shuffle=False
)
```

📋 fit() Parameters

train_gen → training data generator
validation_data → val_gen
epochs → 30 (max training rounds)
callbacks → list of 3 callbacks

⚙️ Callbacks Passed

Checkpoint +generator → saves best model to Drive
early_stop → stops if val_acc stalls (patience=5)
reduce_lr → halves LR if val_acc stalls (patience=3)

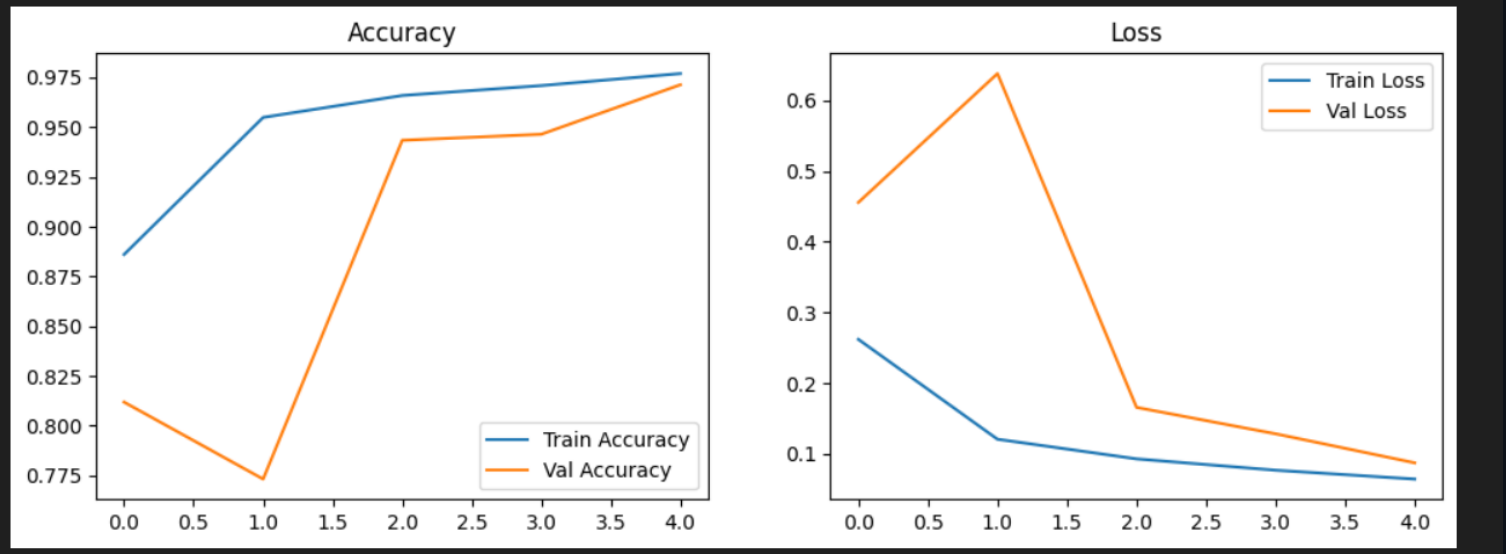
📊 Returns: history object

history.history['accuracy']
history.history['val_accuracy']
history.history['loss']
history.history['val_loss']

1 🚀 Model Training

model.fit() — trains CNN on batches for 5 epochs with callbacks

```
history = model.fit(
  train_gen,
  validation_data=val_gen,
  epochs=30,
  callbacks=[checkpoint, early_stop, reduce_lr]
)
```



📋 fit() Parameters

train_gen → training data generator
validation_data → val_gen
epochs → 30 (max training rounds)
callbacks → list of 3 callbacks

⚙️ Callbacks Passed

checkpoint → saves best model to Drive
early_stop → stops if val_acc stalls (patience=5)
reduce_lr → halves LR if val_acc stalls (patience=3)

📊 Returns: history object

history.history['accuracy']
history.history['val_accuracy']
history.history['loss']
history.history['val_loss']

Model Evaluation

Section 4

TensorFlow | Keras | Sklearn | NumPy | Matplotlib

1 CNN Eval

2 ANN Eval

3 Predictions

4 Confusion

5 Report

6 Results

1 📈 CNN Model Evaluation

Evaluate the CNN on unseen test data — compute accuracy and loss

```python
# CNN Evaluation
test_generator = test_datagen.flow_from_dataframe(
    test_df,
    x_col="image_path", y_col="label",
    target_size=(128,128),
    class_mode="binary",
    batch_size=32, shuffle=False
)

loss, accuracy = model.evaluate(test_generator)
print(f"Test Accuracy: {accuracy*100:.2f}%")
print(f"Test Loss: {loss:.4f}")
```

⚙️ Parameters

test_datagen	rescale=1/255
target_size	(128, 128)
class_mode	"binary"
batch_size	32
shuffle	False

📊 model.evaluate() Returns

```
loss      → float  (e.g. 0.0821)
accuracy  → float  (e.g. 0.9723)
```

🖥️ Expected Output

```
Test Accuracy: 97.23%
Test Loss:     0.0821
```

💡 Why shuffle=False?

Keeps predictions in original order

2 🧠 ANN Model Evaluation

Evaluate ANN, generate predictions and probability scores

```python
# ANN Evaluation
test_loss, test_acc = ann_model.evaluate(
    ann_test_gen, verbose=0)
print(f"Test Loss     : {test_loss:.4f}")
print(f"Test Accuracy : {test_acc:.4f}")

ann_test_gen.reset()

# Generate probability predictions
y_pred_prob = ann_model.predict(ann_test_gen).ravel()
y_pred = (y_pred_prob >= 0.5).astype(int)
y_true = ann_test_gen.classes
class_names = list(ann_test_gen.class_indices.keys())
```

⚙️ Key Steps

`verbose=0`	suppress output
`.reset()`	rewind generator
`.predict()`	raw probabilities
`.ravel()`	flatten to 1D
`>= 0.5`	threshold to 0/1

🔢 Threshold Logic

```
prob >= 0.5  →  1  (diseased)
prob <  0.5  →  0  (healthy)
```

📊 Expected Output

```
Test Loss     : 0.0934
Test Accuracy : 0.9651
```

⚠️ Why .reset()?

Generators must be rewound before

3 📋 Classification Report

Per-class precision, recall, F1-score from sklearn

```python
from sklearn.metrics import classification_report

print(classification_report(
    y_true,
    y_pred,
    target_names=class_names
))

# Sample output:
#               precision  recall  f1
# diseased         0.97    0.96   0.97
# healthy          0.96    0.97   0.96
# accuracy                        0.97
# macro avg        0.97    0.97   0.97
# weighted avg     0.97    0.97   0.97
```

📐 Metrics Explained

```
precision  TP / (TP + FP)
           How many positives
           are truly positive

recall     TP / (TP + FN)
           Of all actual positives,
           how many detected

f1-score   2 * (P*R) / (P+R)
           Harmonic mean of P & R
```

🔑 Arguments

`y_true`	ground truth labels
`y_pred`	model predictions
`target_names`	class labels

💡 Tip

macro avg = unweighted mean
weighted avg = size-adjusted mean

4 🔲 Confusion Matrix — Setup

Build and display the confusion matrix using sklearn and matplotlib

```
from sklearn.metrics import confusion_matrix,\
    ConfusionMatrixDisplay

cm = confusion_matrix(

y_true,

y_pred
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
)

fig, ax = plt.subplots(figsize=(6, 5))
disp.plot(
    ax=ax,
    colorbar=False,
    cmap='Blues'
)
plt.show()
```

🔲 Confusion Matrix Layout

TN True Neg	**FP** False Pos
FN False Neg	**TP** True Pos

↑ Actual

⚙️ Plot Parameters

```
figsize=(6,5)  figure size (inches)
colorbar=False remove sidebar
cmap='Blues'    blue color scale
```

💡 Why Confusion Matrix?

Shows which classes the model confuses — FP vs FN breakdown beyond single accuracy number

5 Full Evaluation Code — CNN

Complete CNN evaluation block — from generator to metrics display

```
# ═══════════════════════════════════════════════
# CNN EVALUATION
# ═══════════════════════════════════════════════
test_generator = test_datagen.flow_from_dataframe(
    test_df, x_col="image_path", y_col="label",
    target_size=(
128,128), class_mode="binary", batch_size=32, shuffle=False
)
loss, accuracy = model.evaluate(test_generator)
print(f"Test Accuracy: {accuracy*100:.2f}%")  print(f"Test Loss: {loss:.4f}")

# ═══════════════════════════════════════════════
# ANN EVALUATION
# ═══════════════════════════════════════════════
test_loss, test_acc = ann_model.evaluate(ann_test_gen, verbose=0)
print(f"Test Loss     : {test_loss:.4f}")  print(f"Test Accuracy : {test_acc:.4f}")
ann_test_gen.reset()
y_pred_prob = ann_model.predict(ann_test_gen).ravel()  y_pred = (y_pred_prob >= 0.5).astype(int)
y_true = ann_test_gen.classes  class_names = list(ann_test_gen.class_indices.keys())
print( classification_report(y_true, y_pred, target_names=class_names) )
cm = confusion_matrix( y_true, y_pred )
disp = ConfusionMatrixDisplay( confusion_matrix=cm, display_labels=class_names )
fig, ax = plt.subplots(figsize=(6, 5))
disp.plot( ax=ax, colorbar=False, cmap='Blues' )  plt.show()
```

6

🏆 Evaluation Results Summary

Side-by-side comparison of CNN vs ANN performance on test set

🖼️ CNN

Convolutional Neural Network

98%

ACCURACY

```
model.evaluate(test_generator)
```

Input: 128×128 images

VS

🧠 ANN

Artificial Neural Network

87%

ACCURACY

```
ann_model.evaluate(ann_test_gen)
```

Input: flattened pixel vectors

🔁 Transfer Learning

Section 5

PlantVillage — ResNet50 Pre-trained Model

Section: Transfer Learning & Fine-Tuning

1 Architecture | 2 Compile | 3 Generators | 4 Callbacks | 5 Training | 6 Evaluation | 7 Comparison

ResNet50 • Python / TensorFlow • Binary Classification

1

Build the ResNet50 Model

Frozen base + custom classification head

```python
# Set image size
img_size   = (128, 128)

# Load ResNet50 — exclude top classifier layer
base_model = ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(128, 128, 3)
)

# Freeze ALL base layers — don't update ImageNet weights
for layer in base_model.layers:
    layer.trainable = False

# Build custom head on top of ResNet50 output
x = base_model.output
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dense(128, activation='relu')(x)
x = tf.keras.layers.Dropout(0.5)(x)
predictions = tf.keras.layers.Dense(1, activation='sigmoid')(x)
resnet_model = Model(inputs=base_model.input, outputs=predictions)
```

Frozen Layers

```
layer.trainable = False
```

→ All ResNet50 conv layers are FROZEN
→ ImageNet weights preserved
→ Only new head is trained

Custom Head

```
ResNet50 output
  ↓ GlobalAveragePooling2D
  ↓ Dense(128, relu)
  ↓ Dropout(0.5)
  ↓ Dense(1, sigmoid)
  = 0 or 1 (binary)
```

2 ⚙️ Compile the Model

Adam optimizer + binary crossentropy + accuracy metric

```python
# Compile ResNet50 model

resnet_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)


resnet_model.summary()
```

⚙️ Compile Settings

```
Optimizer : Adam (adaptive LR)
Loss      : binary_crossentropy
Metrics   : accuracy
```

Adam adapts LR per parameter.
Best for deep networks.

📋 summary()

Prints the full model layers, output shapes, and parameter counts for each layer.

Helps verify architecture before training starts.

🔢 Binary Cross-Entropy

Used for 0/1 output (sigmoid).
Penalizes wrong-confidence.

3 📦 Create Generators

ResNet-specific preprocessing — preprocess_input replaces rescale=1./255

```python
# Import ResNet preprocessing function
from tensorflow.keras.applications.resnet50 import preprocess_input

# ResNet generators — NO rescale, use preprocessing_function instead
resnet_train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)
resnet_val_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)

resnet_train_gen = resnet_train_datagen.flow_from_dataframe(
    dataframe   = train_df,
    x_col       = 'image_path',
    y_col       = 'label',
    target_size = IMG_SIZE,
    batch_size  = BATCH_SIZE,
    class_mode  = 'binary',
    shuffle     = True
)
```

⚠️ preprocess_input vs rescale

ResNet50 was trained with its own normalization (not 0–1).

---

preprocess_input subtracts the ImageNet channel means & scales to [-1, 1] range.

Using rescale=1./255 BREAKS the model's expectations!

🔑 Parameters

dataframe	→ train_df
x_col	→ 'image_path'
y_col	→ 'label'
target_size	→ (128,128)
batch_size	→ 32
class_mode	→ 'binary'
shuffle	→ True (train only)

3

📦 Create Generators

Validation and Test generators — shuffle=False for correct evaluation

```python
resnet_val_gen = resnet_val_datagen.flow_from_dataframe(
    dataframe  = val_df,
    x_col      = 'image_path',
    y_col      = 'label',
    target_size = (128, 128),
    batch_size  = 32,
    class_mode  = 'binary',
    shuffle     = False
)

resnet_test_gen = resnet_test_datagen.flow_from_dataframe(
    dataframe  = test_df,
    x_col      = 'image_path',
    y_col      = 'label',
    target_size = (128, 128),
    batch_size  = 32,
    class_mode  = 'binary',
    shuffle     = False
)

print('Generators ready ✓')
```

⚠️ Why shuffle=False?

Val & Test generators must NOT shuffle.

Prediction order must match y_true order for confusion matrix & report to be correct.

📊 Generator Output

```
resnet_train_gen → train batches
resnet_val_gen   → val batches
resnet_test_gen  → test batches
```

All generators: 128×128×3 preprocessed via preprocess_input (not rescaled to [0,1])

4 Callbacks

EarlyStopping + ReduceLROnPlateau + ModelCheckpoint — prevent overfitting

```python
from tensorflow.keras.callbacks import
    ModelCheckpoint, EarlyStopping, ReduceLROnPlateau

resnet_checkpoint = ModelCheckpoint(
    'best_resnet50_model.h5',
    monitor         = 'val_accuracy',
    save_best_only   = True,
    verbose          = 1
)

resnet_callbacks = [
    resnet_checkpoint,
    EarlyStopping(
        monitor             ='val_accuracy',
        patience            =5,
        restore_best_weights =True, verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',  factor=0.5,
        patience=3,          min_lr=1e-6, verbose=1
    )
]
print('Callbacks ready ✓')
```

EarlyStopping

```
monitor : val_accuracy
patience : 5 epochs
restore_best_weights : True
→  Stops if no improvement
→  Reverts to best weights
```

ReduceLROnPlateau

```
monitor : val_loss
factor  : 0.5  (halves LR)
patience: 3 epochs
min_lr  : 1e-6
→  Cuts LR when loss stalls
```

ModelCheckpoint

```
saves: 'best_resnet50_model.h5'
monitor: val_accuracy
save_best_only: True
→  Keeps only best epoch on disk
```

5 🚀 Train the ResNet50

model.fit() — feeds batches from generator for up to 10 epochs with callbacks

```python
history_resnet = resnet_model.fit(
    resnet_train_gen,
    validation_data = resnet_val_gen,
    epochs          = 10,
    callbacks       = resnet_callbacks
)
```

1. Epoch starts
2. Batch from resnet_train_gen
3. Forward pass → compute loss
4. Backpropagation → update head weights
5. Validate on resnet_val_gen
6. Callbacks check → save / stop / reduce LR
7. Next epoch or early stop

📋 fit() Parameters

```
resnet_train_gen   → train generator
validation_data    → resnet_val_gen
epochs             → 10 (max rounds)
callbacks          → resnet_callbacks
Returns: History object with
accuracy, val_accuracy, loss, val_loss
```

💡 History Object

```
history_resnet.history keys:
  'accuracy'     train acc per epoch
  'val_accuracy' val acc per epoch
  'loss'         train loss
  'val_loss'     validation loss
```

Used to plot training curves.

6 Plot Training History

Side-by-side Accuracy and Loss curves — diagnose overfitting/underfitting

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 5))

# --- Accuracy subplot ---
plt.subplot(1, 2, 1)
plt.plot(history_resnet.history['accuracy'],     label='Train Accuracy')
plt.plot(history_resnet.history['val_accuracy'], label='Val Accuracy')
plt.title('ResNet50 Accuracy')
plt.legend()
plt.grid(True)

# --- Loss subplot ---
plt.subplot(1, 2, 2)
plt.plot(history_resnet.history['loss'],     label='Train Loss')
plt.plot(history_resnet.history['val_loss'],  label='Val Loss')
plt.title('ResNet50 Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

📈 Accuracy Plot

```
X-axis : Epoch number
Y-axis : Accuracy (0 → 1)
Train  : solid line
Val    : dashed line
→  Gap = overfitting
→  Both rising = learning well
```

📉 Loss Plot

```
X-axis : Epoch number
Y-axis : Binary cross-entropy
Train  : decreasing = learning
Val    : should track train loss
→  Val rising = overfitting
```

💡 plt.subplots(1, 2)

```
1 row × 2 col    figsize=(14,5)
axes[0]=acc  axes[1]=loss
```

7 Evaluate & Compare All Models

Loss, Accuracy + side-by-side table: CNN vs ANN vs ResNet50

```python
# Evaluate each model on test set
try:
    cnn_loss,    cnn_acc    = model.evaluate(test_generator,   verbose=0)
    ann_loss,    ann_acc    = ann_model.evaluate(ann_test_gen,    verbose=
    resnet_loss, resnet_acc = resnet_model.evaluate(resnet_test_gen, verbo

    print('\n' + '='*45)
    print(f"{'Model Type':<20} | {'Loss':<10} | {'Accuracy':<10}")
    print('-'*45)
    print(f"{'CNN Custom':<20} | {cnn_loss:<10.4f} | {cnn_acc:<10.4f}")
    print(f"{'ANN Basic':<20} | {ann_loss:<10.4f} | {ann_acc:<10.4f}")
    print(f"{'ResNet50 (TL)':<20} | {resnet_loss:<10.4f} | {resnet_acc:<10
    print('='*45)

except Exception as e:
    print("Make sure to run all model cells first.")
```

🔢 Key Steps

1. evaluate() → loss & accuracy
2. Run all 3 models on test_gen
3. f-string table: aligned cols
4. Loss & Accuracy per model
5. '='*45 for clean borders

📊 Expected Output

```
=============================================
Model Type           | Loss       | Accuracy
---------------------------------------------
```

🏆 Why ResNet50 Wins

Pretrained on 1.2M images.
Reuses learned edge & texture features → higher accuracy with less training time.

Plant Disease Detection

Section 6

Deployment

1 Install | 2 Save Model | 3 Write App | 4 Run | 5 Ngrok | 6 Public URL

PlantVillage Dataset • Python / TensorFlow • Binary Classification

1 📦 Install Required Libraries

Install all dependencies needed for the Streamlit app and model deployment

```
# Install Required Libraries
!pip install streamlit pyngrok tensorflow pillow numpy opencv-python
```

📚 Libraries Installed

streamlit → Web UI framework
pyngrok → Public tunnel (expose localhost)
tensorflow → Load & run trained model
pillow → Image processing (PIL)
numpy → Array operations
opencv-python→ Image decoding (cv2)

💡 Why These Libraries?

→ Streamlit makes web UI with pure Python
→ pyngrok tunnels Colab localhost to web
→ TensorFlow loads the .keras model
→ cv2 decodes uploaded image bytes
→ PIL & numpy handle image array ops

📊 Output

All packages installed in Colab runtime
Ready to run Streamlit app & expose via ngrok

2 💾 Save Trained Model

Export the trained Keras model to disk so Streamlit can load it at runtime

```
# Save Trained Model
model.save("model.keras")
```

🔑 Key Detail

model.save() → Saves the full Keras model
"model.keras" → Native Keras format (v3)
Includes: architecture, weights, optimizer state
File saved to current Colab working directory

📂 What Gets Saved?

✓ Model architecture (layers, filters)
✓ Trained weights (all learned parameters)
✓ Optimizer state (Adam settings)
✓ Training config (loss, metrics)

⚠️ Important Notes

→ Must run AFTER model.fit() completes
→ .keras format (not .h5) — modern standard
→ Used by load_model() in the Streamlit app

📊 Output

model.keras → Saved model file on disk

3 🖋️ Write Streamlit App — Part 1: Imports & Setup

Define the app file and load all required modules and the trained model

```
%%writefile app.py
import streamlit as st
import numpy as np
import cv2
from tensorflow.keras.models import load_model
from PIL import Image

# Load the trained model
model = load_model("model.keras")

# App Title
st.title("🌿 Plant Disease Detection")
st.write("Upload a leaf image for prediction")

# Upload Image
uploaded_file = st.file_uploader(
    "Choose an image",
    type=["jpg", "png", "jpeg"]
)
```

📌 %%writefile app.py

Colab magic command:
→ Writes everything below into app.py
→ Creates the file on disk
→ Streamlit will run this file

🔑 Imports

streamlit → UI components (st.*)
numpy → Array math
cv2 → Image decoding
load_model → Load .keras file
PIL.Image → Image type support

📊 Output

app.py created with UI title & file uploader widget

3 🖌️ Write Streamlit App — Part 2: Image Processing

Read the uploaded image bytes, decode with OpenCV, and display in the app

```
if uploaded_file is not None:
    # Convert uploaded image to OpenCV format
    file_bytes = np.asarray(
        bytearray(uploaded_file.read()),
        dtype=np.uint8
    )
    img_bgr = cv2.imdecode(file_bytes, 1)

    # Check image
    if img_bgr is None:
        st.error("Error loading image")
    else:
        # Convert BGR → RGB
        img_rgb = cv2.cvtColor(
            img_bgr,
            cv2.COLOR_BGR2RGB
        )
        # Display uploaded image
        st.image(
            img_rgb,
            caption="Uploaded Image",
            use_container_width=True
        )
```

🔄 Processing Pipeline

1. uploaded_file.read() → raw bytes
2. bytearray() → byte array
3. np.asarray()→ numpy uint8 array
4. cv2.imdecode()→ decode to BGR image
5. cvtColor BGR→RGB for display

⚠️ BGR vs RGB

OpenCV reads images as BGR
Streamlit/matplotlib expect RGB
→ cvtColor converts the order
→ Skipping causes wrong colors

📊 Output

Image displayed inside the Streamlit app

3 🖌️ Write Streamlit App — Part 3: Preprocess & Predict

Resize, normalize, expand dims — then run model.predict() and classify the result

```
# Image Preprocessing
img_resized = cv2.resize(
    img_rgb,
    (128, 128)
)
img_normalized = img_resized / 255.0
img_final = np.expand_dims(
    img_normalized,
    axis=0
)

# Prediction
prediction = model.predict(
    img_final,
    verbose=0
)
prob = prediction[0][0]

# Classification Logic
if prob <= 0.5:
    label = "Diseased"
    confidence = prob
    st.error(
        f"Prediction: {label} 😷🌱"
    )
    st.write(
        f"Confidence: {confidence:.2f}"
    )
else:
    label = "Healthy"
    confidence = 1 - prob
```

⚙️ Preprocessing Steps

resize(128,128) → match training size
/ 255.0 → normalize to [0,1]
expand_dims(,0) → add batch dim
Shape: (128,128,3) → (1,128,128,3)

🎯 Classification Logic

prob = sigmoid output (0.0–1.0)
prob ≤ 0.5 → Diseased 😷
prob > 0.5 → Healthy 🌱
confidence shown as float :.2f
st.error() → red box (diseased)
st.success()→ green box (healthy)

4 ▶️ Start Streamlit App

Launch the Streamlit server on port 8501 in the Colab background

```
# Start Streamlit App
get_ipython().system_raw(
    "streamlit run app.py --server.port 8501 &"
)
```

```
# Ngrok Authentication
!ngrok authtoken YOUR_NGROK_TOKEN
```

```
# Create Public URL
from pyngrok import ngrok
public_url = ngrok.connect(8501)
print("Public URL:", public_url)
```

🔧 system_raw()

Runs shell command in background (&)
Non-blocking → cell continues
--server.port 8501 → fixed port
app.py is the Streamlit script

🔑 Ngrok Auth Token

→ Sign up at ngrok.com (free)
→ Get your authtoken
→ Replace YOUR_NGROK_TOKEN
→ Required to create tunnels

🌐 ngrok.connect(8501)

Creates a secure public HTTPS URL
Tunnels traffic → localhost:8501
Share the URL with anyone globally
Output: https://xxxx.ngrok.io

✅ 📄 Full Code — Complete Deployment

Every single line — nothing deleted — ready to run cell by cell

```
# Install Required Libraries
!pip install streamlit pyngrok tensorflow pillow numpy opencv-
python

# Save Trained Model
model.save("model.keras")

# Write Streamlit App
%%writefile app.py
import streamlit as st
import numpy as np
import cv2
from tensorflow.keras.models import load_model
from PIL import Image

# Load the trained model
model = load_model("model.keras")

# App Title
st.title("🌿 Plant Disease Detection")
st.write("Upload a leaf image for prediction")

# Upload Image
uploaded_file = st.file_uploader(
    "Choose an image",
    type=["jpg", "png", "jpeg"]
)
```

```
if uploaded_file is not None:
    file_bytes = np.asarray(
        bytearray(uploaded_file.read()),
        dtype=np.uint8
    )
    img_bgr = cv2.imdecode(file_bytes, 1)
    if img_bgr is None:
        st.error("Error loading image")
    else:
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
        st.image(img_rgb, caption="Uploaded Image",
                 use_container_width=True)
        img_resized = cv2.resize(img_rgb, (128, 128))
        img_normalized = img_resized / 255.0
        img_final = np.expand_dims(img_normalized, axis=0)
        prediction = model.predict(img_final, verbose=0)
        prob = prediction[0][0]
        if prob <= 0.5:
            label = "Diseased"
            confidence = prob
            st.error(f"Prediction: {label} 😳🌱")
            st.write(f"Confidence: {confidence:.2f}")
        else:
            label = "Healthy"
            confidence = 1 - prob
            st.success(f"Prediction: {label} 🌱")
            st.write(f"Confidence: {confidence:.2f}")

# Start Streamlit App
get_ipython().system_raw(
    "streamlit run app.py --server.port 8501 &")
```

Deployment Summary

End-to-end overview of the 6-step deployment pipeline

1 — Install Libraries

```
pip install streamlit pyngrok tensorflow pillow numpy opencv-python
```

2 — Save Model

```
model.save("model.keras")  → Export trained weights to disk
```

3 — Write App

```
%%writefile app.py  → Full Streamlit UI with upload & predict
```

4 — Run App

```
system_raw streamlit run app.py --server.port 8501 &
```

5 — Ngrok Auth

```
!ngrok authtoken YOUR_TOKEN  → Authenticate with ngrok service
```

6 — Public URL

```
ngrok.connect(8501)  → Get shareable HTTPS URL for the app
```

ANN Model

PlantVillage — Healthy vs Diseased

Section: Model Training & Architecture

1 Generators | 2 Architecture | 3 Callbacks | 4 Training | 5 Visualization | 6 Evaluation | 7 Comparison

```
# ANN PlantVillage

model = Sequential()

model.add(Flatten())

model.add(Dense(512))

model.add(Dropout(0.4))

model.add(Dense(1,

  activation='sigmoid'))

model.compile(

  optimizer=Adam(),

  loss='binary_crossentropy'

)
```

1 📦 Create Generators

Batch loading via flow_from_dataframe — no RAM crash

```python
# Reuse the same datagens from your original notebook
# train_datagen, val_datagen, test_datagen already have rescale=1./255

BATCH_SIZE = 32
IMG_SIZE   = (128, 128)

ann_train_gen = train_datagen.flow_from_dataframe(
    dataframe   = train_df,
    x_col       = 'image_path',
    y_col       = 'label',
    target_size = IMG_SIZE,
    batch_size  = BATCH_SIZE,
    class_mode  = 'binary',
    shuffle     = True
)
```

📁 What it does

→ Reuses datagens from preprocessing nb
→ Sets BATCH=32, IMG_SIZE=(128,128)
→ Loads images lazily (batch-by-batch)
→ No RAM overflow risk
→ Binary class_mode: 0=healthy, 1=diseased

🔑 Parameters

dataframe → train_df / val_df / test_df
x_col → 'image_path'
y_col → 'label'
target_size → (128,128)
class_mode → 'binary'

1 📦 Create Generators

Validation and Test generators — shuffle=False for correct evaluation

```python
ann_val_gen = val_datagen.flow_from_dataframe(
    dataframe   = val_df,
    x_col       = 'image_path',
    y_col       = 'label',
    target_size = IMG_SIZE,
    batch_size  = BATCH_SIZE,
    class_mode  = 'binary',
    shuffle     = False
)

ann_test_gen = test_datagen.flow_from_dataframe(
    dataframe   = test_df,
    x_col       = 'image_path',
    y_col       = 'label',
    target_size = IMG_SIZE,
    batch_size  = BATCH_SIZE,
    class_mode  = 'binary',
    shuffle     = False
)

print('Generators ready ✓')
```

⚠️ Why shuffle=False?

Val & Test generators must NOT shuffle.
Prediction order must match y_true order
for confusion matrix & classification
report to be correct.

📊 Generator Output

ann_train_gen → training batches
ann_val_gen → validation batches
ann_test_gen → test batches

All generators: 128×128×3 RGB images
rescaled to [0,1] range

2 🧠 Build the ANN Model

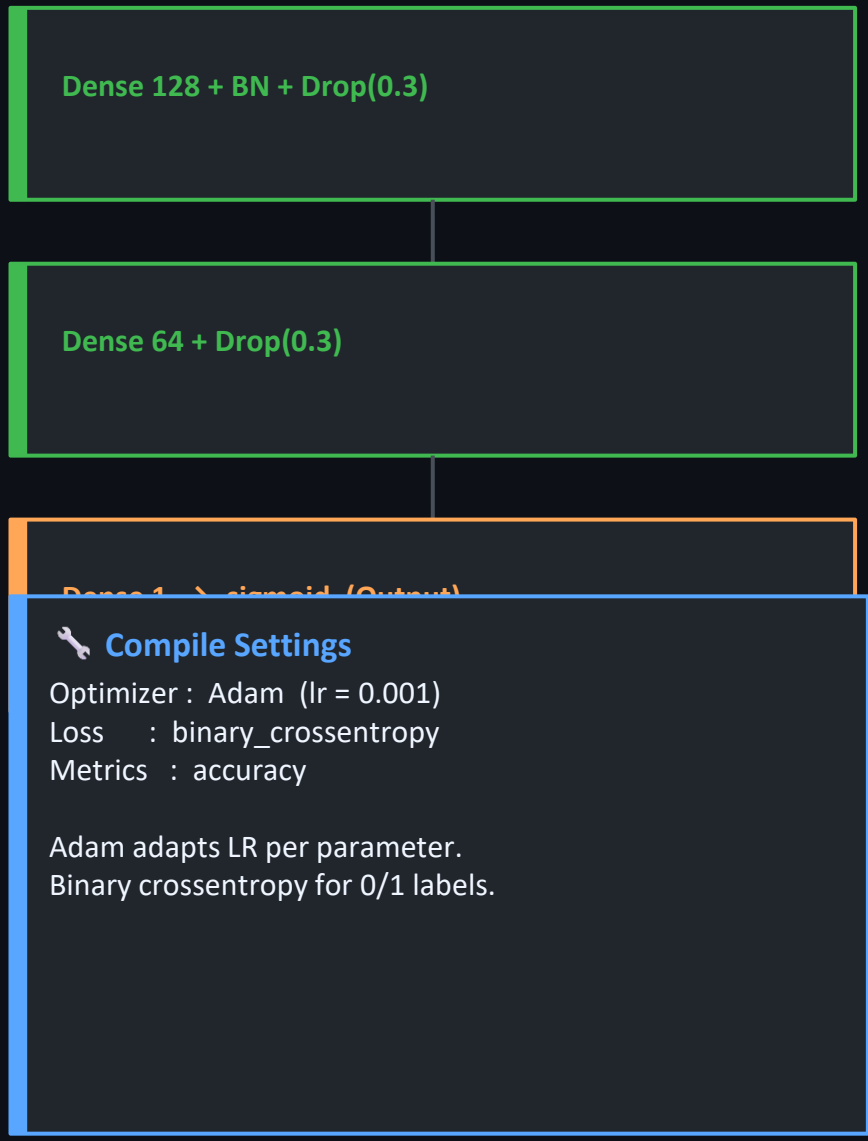
Flatten layer is INSIDE the model — images flattened batch-by-batch on GPU, never all at once in RAM

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Flatten, Input
from tensorflow.keras.optimizers import Adam

ann_model = Sequential(name='ANN_PlantVillage')

# Flatten 128x128x3 image -> 49152 vector (done inside model, per batch)
ann_model.add(Input(shape=(128, 128, 3)))
ann_model.add(Flatten())

# Hidden layer 1
ann_model.add(Dense(512, activation='relu'))
ann_model.add(BatchNormalization())
ann_model.add(Dropout(0.4))

# Hidden layer 2
ann_model.add(Dense(256, activation='relu'))
ann_model.add(BatchNormalization())
ann_model.add(Dropout(0.4))
```

2 🧠 Build the ANN Model (continued)

Hidden layers 3 & 4 + Output layer + Compile

```python
# Hidden layer 3
ann_model.add(Dense(128, activation='relu'))
ann_model.add(BatchNormalization())
ann_model.add(Dropout(0.3))

# Hidden layer 4
ann_model.add(Dense(64, activation='relu'))
ann_model.add(Dropout(0.3))

# Output
ann_model.add(Dense(1, activation='sigmoid'))

ann_model.compile(
    optimizer = Adam(learning_rate=1e-3),
    loss      = 'binary_crossentropy',
    metrics   = ['accuracy']
)

ann_model.summary()
```



3 ⚙️ Callbacks

EarlyStopping + ReduceLROnPlateau + ModelCheckpoint — prevent overfitting and save best model

```python
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

ann_callbacks = [
    EarlyStopping(
        monitor='val_accuracy',
        patience=5,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
        min_lr=1e-6,
        verbose=1
    ),
    ModelCheckpoint(
        'best_ann_model.h5',
        monitor='val_accuracy',
        save_best_only=True,
        verbose=1
    )
]

print('Callbacks ready ✓')
```

🛑 EarlyStopping

monitor : val_accuracy
patience : 5 epochs
restore_best_weights : True
→ Stops if no improvement for 5 epochs
→ Reverts to best-seen weights

📉 ReduceLROnPlateau

monitor : val_loss
factor : 0.5 (halves LR)
patience: 3 epochs
min_lr : 1e-6
→ Cuts LR when loss stalls

💾 ModelCheckpoint

saves: 'best_ann_model.h5'
monitor: val_accuracy
save_best_only: True
→ Keeps only best epoch on disk

4 🚀 Train the ANN

model.fit() — feeds batches from generator for up to 30 epochs with callbacks

```
ann_history = ann_model.fit(
    ann_train_gen,
    validation_data = ann_val_gen,
    epochs          = 30,
    callbacks       = ann_callbacks
)
```

1. Epoch starts
2. Batch from train_gen
3. Forward pass → loss
4. Backprop → update weights
5. Validate on val_gen
6. Callbacks check metrics
7. Next epoch or early stop

📋 fit() Parameters

ann_train_gen → training data generator
validation_data→ ann_val_gen
epochs → 30 (max, may stop early)
callbacks → ann_callbacks list

Returns: History object with
accuracy, val_accuracy, loss, val_loss

💡 History Object

ann_history.history keys:
'accuracy' train acc per epoch
'val_accuracy' val acc per epoch
'loss' train loss
'val_loss' validation loss

5 📊 Plot Training History

Side-by-side Accuracy and Loss curves — diagnose overfitting/underfitting

```python
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(ann_history.history['accuracy'],     label='Train')
axes[0].plot(ann_history.history['val_accuracy'], label='Validation')
axes[0].set_title('ANN — Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(ann_history.history['loss'],     label='Train')
axes[1].plot(ann_history.history['val_loss'], label='Validation')
axes[1].set_title('ANN — Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

📈 Accuracy Plot

X-axis : Epoch number
Y-axis : Accuracy (0 → 1)
Train : solid line
Val : dashed line
→ Gap = overfitting
→ Both rising = learning well

📉 Loss Plot

X-axis : Epoch number
Y-axis : Binary cross-entropy
Train : decreasing = learning
Val : should track train loss
→ Val rising = overfitting
→ Both flat = stopped learning

💡 plt.subplots(1, 2)

Creates 1 row × 2 column figure.
axes[0] = accuracy, axes[1] = loss

6 🧪 Evaluate on Test Set

Loss, Accuracy, Classification Report & Confusion Matrix

```python
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import numpy as np

# Loss & Accuracy
test_loss, test_acc = ann_model.evaluate(ann_test_gen, verbose=0)
print(f'Test Loss     : {test_loss:.4f}')
print(f'Test Accuracy : {test_acc:.4f}')

# Predictions
ann_test_gen.reset()
y_pred_prob = ann_model.predict(ann_test_gen).ravel()
y_pred      = (y_pred_prob >= 0.5).astype(int)
y_true      = ann_test_gen.classes

class_names = list(ann_test_gen.class_indices.keys())

print('\nClassification Report:')
print(classification_report(y_true, y_pred, target_names=class_names))
```

🔢 Key steps

1. evaluate() → loss & accuracy
2. reset() generator before predict
3. predict() → sigmoid probabilities
4. threshold 0.5 → binary classes
5. .classes → true labels (ordered)
6. classification_report() → P/R/F1

⚠️ Why reset() before predict?

Generator keeps an internal pointer. After evaluate(), pointer may be mid-stream. reset() ensures predictions start from image index 0 — matching y_true = ann_test_gen.classes order.

📊 Threshold Logic

(y_pred_prob >= 0.5).astype(int)
sigmoid output ≥ 0.5 → class 1

6 🧪 Evaluate — Confusion Matrix

Visual confusion matrix using ConfusionMatrixDisplay with Blues colormap

```
# Confusion Matrix
cm   = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=class_names)
fig, ax = plt.subplots(figsize=(6, 5))
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title('ANN — Confusion Matrix (Test Set)')
plt.tight_layout()
plt.show()
```

TP = True Positive (healthy → healthy)
TN = True Negative (diseased → diseased)
FP = False Positive (diseased → healthy)
FN = False Negative (healthy → diseased)